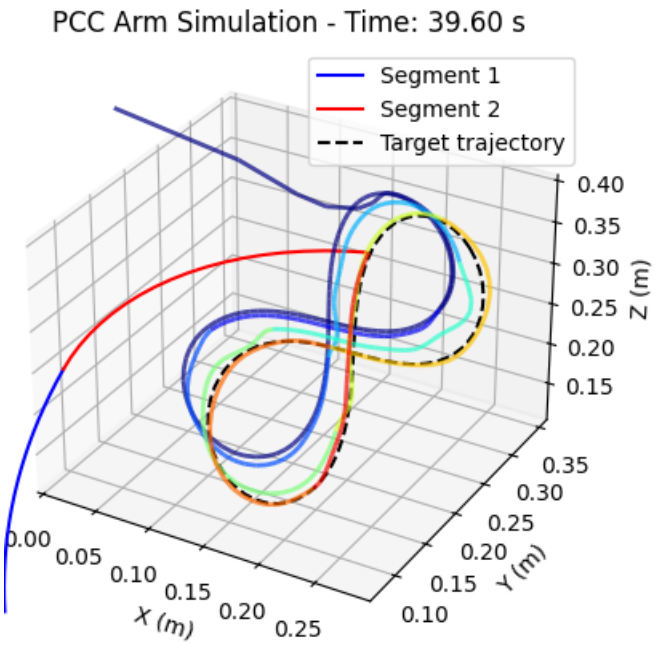
École Polytechnique Fédérale de Lausanne

Martin Daigne 340936

Data-Enabled Model Predictive Control for Compliant Manipulators



January 02, 2026

CREATE Lab Semester Project

Contents

1 Introduction **2**
1.1 Previous work . . . 2
1.2 This work . . . 2

2 Methods **2**
2.1 Kinematics . . . 3
2.2 Dynamics . . . 3
2.3 Control Formulation . . . 3

3 Experimental Setup **4**
3.1 Parameter selection . . . 4
3.2 Hardware . . . 4

4 Results **4**
4.1 Different Trajectories . . . 5
4.2 Convergence analysis . . . 5
4.3 Real arm implementation . . . 7

5 Discussion & Conclusion **7**

A Kinematics **9**

B Dynamics **9**

C Code **9**

1 Introduction

Dynamic control of compliant manipulators is still an open challenge, with many of the current issues arising from the high degree of nonlinearities in system dynamics. Whilst the use of Model Predictive Control (MPC) has been increasing in conjunction with computing capabilities, the uptake in the space of compliant manipulators has been constrained by the inability to efficiently and effectively solve the optimization problem online using methods suitable for real-world deployment. This project will investigate these factors and aim to develop a fast, reliable and robust MPC framework for compliant manipulators. The overarching aim is to be able to modify the system dynamics online when accounting for factors such as operating environment (water/air) and hysteresis effects.

1.1 Previous work

Prior work on MPC for continuum and soft manipulators often assumes either (i) accurate model identification offline, (ii) slow dynamics where computation time is less critical, or (iii) learning-based residual models that require large training datasets and may generalize poorly outside the training domain.

There is also Moving Horizon Estimation (MHE) formulations [4] that provide theoretical robustness but often needs too much computation for high-frequency control loops. Alternatively, Model Reference Adaptive Control (MRAC) [5] has been explored but can struggle with the explicit constraint handling required for tendon-driven systems.

In contrast, this project focuses on a lightweight approach that can be implemented with minimal data: a moving-window parameter adaptation that updates a small set of physically interpretable parameters (mass, damping, bending stiffness) directly from recent state-input history, and integrates these updates into the MPC loop.

1.2 This work

The main contributions are: (1) a Piecewise Constant Curvature (PCC)-based nonlinear dynamic model including tendon actuation mapping and optional fluid effects; (2) a constrained nonlinear MPC formulation in acados using a nonlinear least-squares tracking cost on end-effector position and joint velocities; and (3) an online parameter adaptation routine that estimates additive corrections to mass, damping, and stiffness from a short history window. The resulting closed-loop system is evaluated on end-effector trajectories and analyzed in terms of tracking error and computational timing.

2 Methods

Notation:

$$\begin{cases} q := [\phi_1, \theta_1, ..., \phi_n, \theta_n],\ \theta_i \text{ bending angle of segment } i,\ \phi_i \text{ angle at the base of segment } i \\ x := [q, \dot{q}] \\ p := \text{adaptive parameters} \\ u := \text{input} \\ \tau := \text{generalized torques} \\ \tau_{\text{tendon}} := \text{tendon tension} \end{cases}$$

The proposed solution to the problem is to build a classic nonlinear-MPC (NMPC) based on some dynamic model $\dot{x} = f(x, u, p) \Leftrightarrow x_+ = F(x, u, p)$, where f is the dynamical model and F the discrete-time integrator. At each timestep, we receive new state measurements x_{meas}, update the adaptive parameters through a least-squares minimization $p = \underset{p}{\text{argmin}} \sum_{i=k-N_{\text{prev}}}^{k-1} \|x_{i+1}^{meas} - F(x_i^{meas}, u_i; p)\|_2^2 + \lambda \|p - p_{\text{prev}}\|^2$ and then perform the MPC update to get the control input using the updated parameters. λ is a regularization term penalizing rapid parameter fluctuation.

The problem is non-convex so to ensure meaningful minima, the parameters are bound into a physically meaningful region (see 2.2 positive mass $m + m_a > 0$, stiffness $k + k_a > 0$ and damping $\beta + \beta_a > 0$) with a 0 initial value (as if there was no mismatch).

The adaptive MPC loop is represented in Algorithm 1.

Algorithm 1 Adaptive NMPC loop

for $k = 0, 1, \ldots, T_{\text{sim}}/\Delta t - 1$ **do**
  (Reference assembly) Build $X_{k:k+N}^{\text{task space, ref}} \leftarrow \left[x_k^{\text{task space, ref}}, \ldots, x_{k+N}^{\text{task space, ref}}\right]$
  (NMPC solve) $\tau_{\text{tendon, k}} \leftarrow \text{acados}(\hat{x}_k, X_{k:k+N}^{\text{task space, ref}}, p_k)$
  (Apply to true system) $x_{k+1}^{\text{true}} \leftarrow f_{\text{true}}(x_k^{\text{true}}, \tau_{\text{tendon, k}})$
  (Measurement update) $\hat{x}_{k+1} \leftarrow x_{k+1}^{\text{true}} + \varepsilon_{k+1}$ ▷ add measurement noise
  (Parameter adaptation) $p_{k+1} \leftarrow p_k$
  Solve Ipopt update:
  $p_{k+1} = \underset{p}{\text{argmin}} \sum_{i=k-N_{\text{prev}}}^{k-1} \|x_{i+1}^{meas} - F(x_i^{meas}, \tau_{\text{tendon, i}}; p)\|_2^2 + \lambda \|p - p_{\text{prev}}\|^2$
end for

2.1 Kinematics

A simple kinematics model which has been proven accurate [3, 6] is considering PCC. The kinematics of each segment has only its length l_i as parameter to be measured. We can chain the kinematic transformation to get the kinematics for a multi segment arm. The full derivation can be found in appendix A.

2.2 Dynamics

The dynamics are derived from the task space equations and transformed into the joint space using the kinematics. They are $M(q)\ddot{q} + (C(q, \dot{q}) + D(q, \dot{q}))\dot{q} + Kq + G(q) = \tau$ with τ the generalized forces. The arm is tendon-driven so in the end we need tendon tension. Initially, a simple Quadratic Programming (QP) to map the generalized forces to tendon tensions was used but has been removed for a better approach: the tendon tensions $\tau_{\text{tendon}} \in \mathbb{R}^{3n}$ are directly optimized upon via $\tau = J_{\text{tendon}}(q)^T \tau_{\text{tendon}}$ with box constraints $0 \leq \tau_{\text{tendon}} \leq \tau_{\text{tendon}}^{\max}$. A broken tendon can be simulated by setting its upper bound to 0.

The stiffness and damping are chosen to be diagonal because they are decoupled: $K = \begin{pmatrix} K_1 & 0 \\ 0 & K_2 \end{pmatrix}$ with $K_i = \begin{pmatrix} k_{\phi_i} + k_{\phi_i,a} & 0 \\ 0 & k_{\theta_i} + k_{\theta_i,a} \end{pmatrix}$ the ith segment spring matrix and $D = \begin{pmatrix} (\beta_1 + \beta_{1,a})K_1 & 0 \\ 0 & (\beta_2 + \beta_{2,a})K_2 \end{pmatrix}$ constants.

The adaptive parameters per segment are $p_a = (m_{i,a}, \beta_{i,a}, k_{\phi_i,a}, k_{\theta_i,a})$.

For simplicity we use proportional damping, $D = \beta K$, which is consistent with a Kelvin–Voigt-type viscoelasticity model. This keeps the parameter set small while capturing the dominant dissipative behavior. The full dynamics derivation can be found in appendix B.

2.3 Control Formulation

The proposed framework follows a classic NMPC formulation 1 as it has been studied thoroughly and work has been done to make it fast through open-source libraries such as acados.[8]. To further reduce computational load, we use a frozen-coefficient approximation over the prediction horizon: M,C,G are evaluated at the horizon start $(q_0, \dot{q}_0)$ and held constant during integration over the horizon. This reduces cost but can degrade stability/accuracy in strongly nonlinear regimes. For example, when the arm is down, the gravitational term will inject energy making the system unstable. This is tackled by unfreezing the gravitational term for example.

The hysteresis or environment change are not disturbances changing every iteration. This means the solver does not need to finish finding the best parameters at each time step. Instead, we can put a time constraint on the solver and warm starting it. The solver won't find the optimum at each step but will converge to it. If we increase the NMPC frequency we can adapt the solver time constraint and it will only need more steps to converge while complying with the NMPC loop frequency.

To follow a trajectory in the task space, the kinematics of the end effector is used to map the states to the end effector position $x_k \in \mathbb{R}^{3n} \rightarrow x_k^{\text{task space}} \in \mathbb{R}^3$. There is no state constraint ($\mathcal{X} = \mathbb{R}^{4n}$) and tendon tensions

are constrained to be positive and less than the motors capability.

$$
\begin{aligned}
\underset{\tau_{\text{tendon}}}{\text{minimize}} \quad & \sum_{k=0}^{N-1} (x_k^{\text{task space}} - x_k^{\text{task space, ref}}) Q (x_k^{\text{task space}} - x_k^{\text{task space, ref}}) + (\tau_{\text{tendon, k}} - \tau_{\text{tendon, k}}^{\text{ref}}) R (\tau_{\text{tendon, k}} - \tau_{\text{tendon, k}}^{\text{ref}}) \\
\text{subject to} \quad & x_{k+1} = f(x_k, \tau_{\text{tendon, k}}, p), \quad k = 0, \ldots, N-1, \\
& x_0 = x(t), \\
& x_k \in \mathcal{X}, \\
& \tau_{\text{tendon, k}} \in \mathcal{U}
\end{aligned}
\tag{1}
$$

3 Experimental Setup

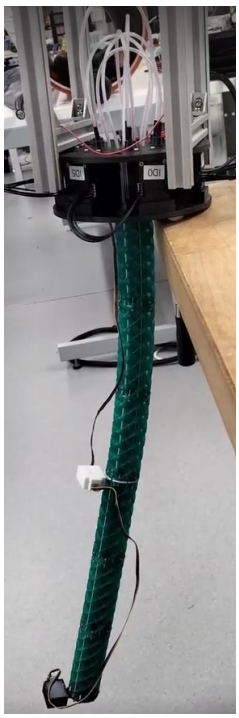


Figure 1: Upside down arm with IMUs on the segments' ends

We will be using the robotic arm used in [2], fig 1 but increasing the aspect ratio to amplify dynamic effects to have more significant errors for better demonstration. The arm has the particularity to be almost incompressible in compression and traction but it can bend. For simplicity we focused on the 2-segment case but the code has been implemented for n-segments and experiments were performed for 2 and 3 segments.

The MPC was set up in acados [8], the adaptive solver in CasADi [1] using IPOPT [9]. A few trajectories are possible to generate (circle, rectangle or lemniscate) and the arm's base can be either pointing up or down.

There are a few model mismatches implemented: "air", "water" (with added mass, and added drag proportional to the velocity squared) or "broken tendon". Hysteresis is slow-varying: the adaptation method trivially tracks slow drift in sim. Therefore we focus on the harder abrupt mismatch (fluid drag/add mass). Hysteresis is more interesting to try on the real arm operating over a long period. The simulation's integrator is an Explicit Runge-Kutta 4 while the MPC integrator is an Implicit Runge-Kutta method for more robustness.

For a more realistic simulation, white noise has been added with standard deviation $\sigma_\theta = \sigma_\phi = 1°$ (angles) and $\sigma_{\dot\theta} = \sigma_{\dot\phi} = 2.5°/s$ (angular rates) to approximate the BNO085 sensor characteristics used on the real arm to get the robot's states.

For the implementation on the arm, tension is mapped to torque, with the pulley radius, which is then mapped to current with the motor's datasheet [7]: $I = a(\tau_{\text{tendon}} \cdot r_{\text{pulley}}) + b$

3.1 Parameter selection

The initial parameters of the system p were chosen as follows: the mass m was measured, the stiffness k was tuned so that, with the arm pointing upward, the rest configuration matches the observed bending angle and the damping coefficient β was tuned so that the free oscillation about the rest configuration matches the observed decay.

3.2 Hardware

All the code was run using a 12th Gen Intel(R) Core(TM) i7-1260p 2.10Ghz processor on a windows machine using WSL for C compiling. The motors are the Dynamixel XM430-W350 [7]. The robot arm in the upside down configuration is shown in figure 1.

4 Results

In fig 2 we see that the program adapted correctly to the environmental change in less than 1 second. For this experiment, the controller is initialized with the model in air while the true system updates its state with the water model.

Because the dynamics are time-varying, it makes more sense to look at the loop average which is an order of magnitude lower after the adaptation.

For the next results, the adaptation will only be done once every loop to better see the effect over time. Also, all results are done with the environment change where the MPC thinks it is in air while the simulation

is based on water. This is more challenging than hysteresis (which is a slow change of D and K) because it depends on the square of the velocity which is not directly tackled by our model.

(a) Trajectory

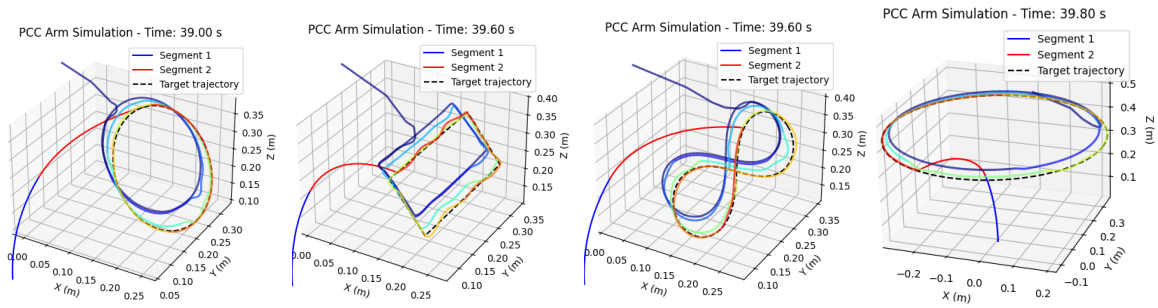
(b) Error over time, in red the optimization steps

(c) Compute time

Figure 2: 2 segment arm in air/water adaptation

4.1 Different Trajectories

Three different trajectories are implemented: a circle, a rectangle and a lemniscate. They are shown in figure 3.



(a) Circle trajectory (b) Rectangle trajectory (c) Lemniscate trajectory (d) Globe circle trajectory

Figure 3: 2 segment arm in air/water adaptation following various trajectories. Each color in the tip trajectory is for a different set of adaptive parameters.

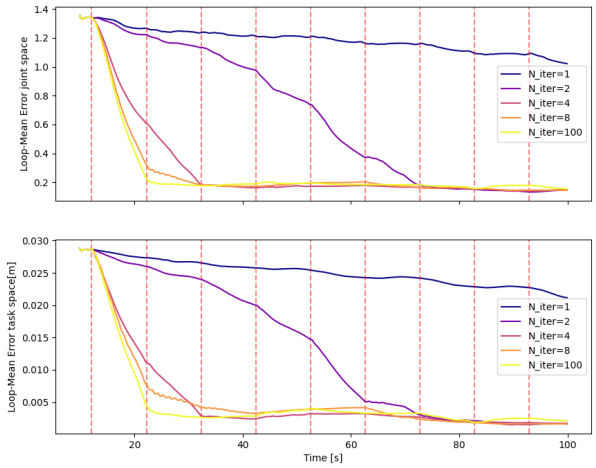
As shown in figure 3, the controller adapts correctly to all trajectories, even the ones with sharp turns.

4.2 Convergence analysis

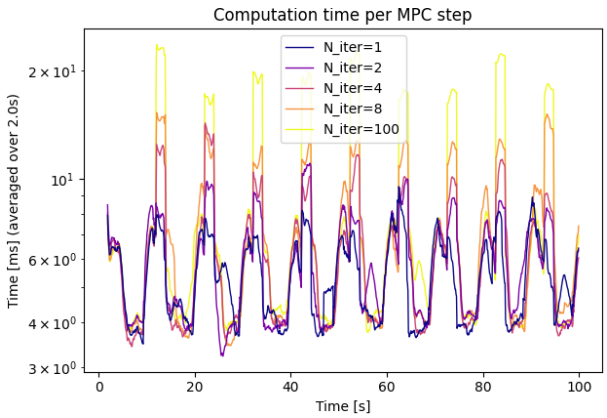
The convergence rate depends on a few factors such as noise, the number of iteration of the solver or the number of points taken into account for the adaptation. On figure 4a, as expected, the more iteration the solver has to find the best parameters, the faster it finds the optimal solution. However, each step takes longer computation time.

Similarly on figure 4c, we see that the more previous points we give to the solver, the faster it finds the optimal solution but each step takes longer computation time. However, when fewer points are used, we observed that the solution is less stable as it fits only the local error which is time varying. The arm oscillates around the trajectory.

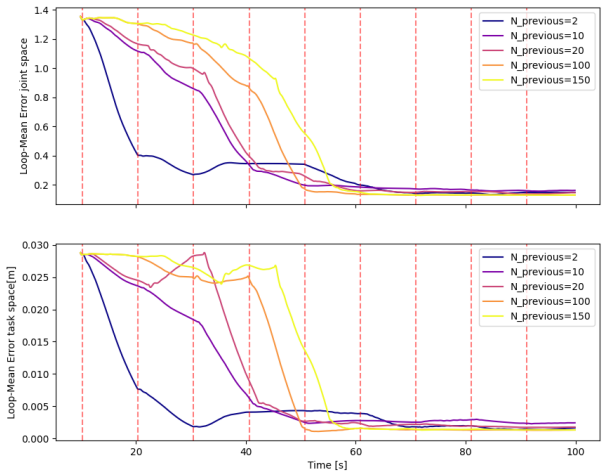
With noise, the adaptation still works, see figures 4e, 4f . Moreover, using more previous points for the adaptations improves the performance because the adaptive solver error is proportional to the averaged error which is an unbiased estimate of the noise-free error. The more points we have, the better the estimation of the noise-free error.



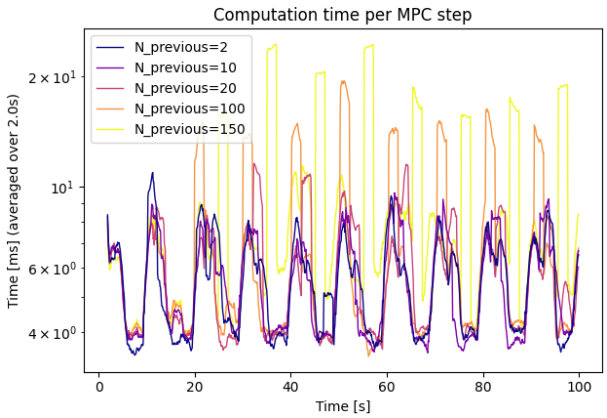
(a) Error over time, with varying number of optimization steps, in red the optimization steps



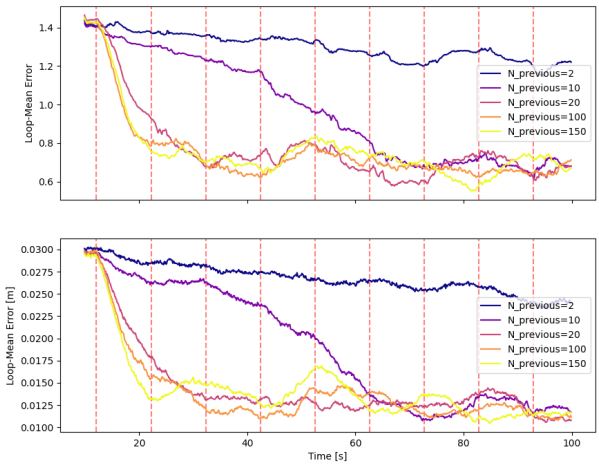
(b) Compute time, with varying number of optimization steps



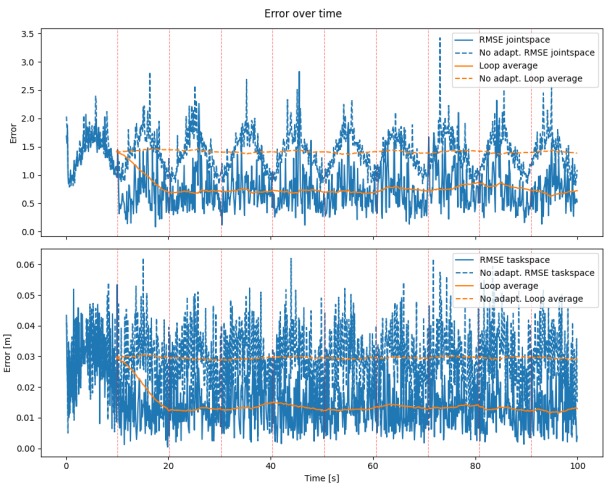
(c) Error over time, with varying number of previous points for the optimization, in red the optimization steps



(d) Compute time, with varying number of previous points for the optimization



(e) With noise, error over time, with varying number of previous points for the optimization, in red the optimization steps



(f) With noise, error over time, with 20 previous points for the optimization, in red the optimization steps

Figure 4: Effect of number of optimization steps and previous points on convergence.

4.3 Real arm implementation

Open-loop without adaptation failed initially due to the tendon tubes buckling. After a fix, the arm moves but sequentially with a lot of delay and with weird behavior (all tendon pulled blocks the robot instead of pulling it to the straight configuration).

5 Discussion & Conclusion

Performing a decoupled sliding-window least-squares update of model parameters using measured states, proved to be a fast and sufficient method in the sense that it does not require any data to be collected beforehand nor does it require the complexity of full MHE or neural network-based residuals. However, it is not always robust to noise as it can quickly overfit local errors. Weights on the parameters' size or rate of change helps but is not sufficient, good measurements or estimates are necessary.

The convergence's rate is also limited by near-singularities happening when a segment is close to straight. This makes the Hessian very spiky and slows both the MPC and the adaptive solver. Future work could use different parametrization for the kinematics [3].

Learning residuals through a neural network would bring more robustness but would require data to train so the robot does not really go into "unseen environment". This method has the advantage of working in completely unseen environments.

MHE, in contrast, is an optimization-based estimator that jointly reconstructs the state trajectory and parameters over a window by balancing measurement residuals and model-consistency residuals, making it more robust to noisy/partial sensing and better suited for constraint handling but is computationally heavier which may be tackled by leveraging acados.

References

[1] Joel A E Andersson et al. "CasADi – A software framework for nonlinear optimization and optimal control". In: *Mathematical Programming Computation* 11.1 (2019), pp. 1–36. DOI: 10.1007/s12532-018-0139-4.

[2] Zixi Chen et al. "A Versatile Neural Network Configuration Space Planning and Control Strategy for Modular Soft Robot Arms". In: *IEEE Transactions on Robotics* 41 (2025), pp. 4269–4282. ISSN: 1941-0468. DOI: 10.1109/tro.2025.3582807. URL: http://dx.doi.org/10.1109/TRO.2025.3582807.

[3] Cosimo Della Santina, Christian Duriez, and Daniela Rus. "Model-Based Control of Soft Robots: A Survey of the State of the Art and Open Challenges". In: *IEEE Control Systems* 43.3 (June 2023), pp. 30–65. ISSN: 1941-000X. DOI: 10.1109/mcs.2023.3253419. URL: http://dx.doi.org/10.1109/MCS.2023.3253419.

[4] Emilien Flayac, Girish Nair, and Iman Shames. "Nonlinear Dual control based on Fast Moving Horizon estimation and Model Predictive Control with an observability constraint". In: *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, Dec. 2021, pp. 3030–3035. DOI: 10.1109/cdc45484.2021.9683222. URL: http://dx.doi.org/10.1109/CDC45484.2021.9683222.

[5] Phillip Hyatt, Curtis C. Johnson, and Marc D. Killpack. "Model Reference Predictive Adaptive Control for Large-Scale Soft Robots". In: *Frontiers in Robotics and AI* 7 (Oct. 2020), p. 558027. ISSN: 2296-9144. DOI: 10.3389/frobt.2020.558027. URL: https://www.frontiersin.org/article/10.3389/frobt.2020.558027/full (visited on 12/20/2025).

[6] Priyanka Rao et al. "How to Model Tendon-Driven Continuum Robots and Benchmark Modelling Performance". In: *Frontiers in Robotics and AI* Volume 7 - 2020 (2021). ISSN: 2296-9144. DOI: 10.3389/frobt.2020.630245. URL: https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2020.630245.

[7] ROBOTIS. *Dynamixel XM430-W350 Datasheet*. ROBOTIS E-Manual. URL: https://emanual.robotis.com/docs/en/dxl/x/xm430-w350/ (visited on 01/02/2026).

[8] Robin Verschueren et al. "acados – a modular open-source framework for fast embedded optimal control". In: *Mathematical Programming Computation* (2021).

[9] Andreas Wächter and Lorenz T. Biegler. "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". en. In: *Mathematical Programming* 106.1 (Mar. 2006), pp. 25–57. ISSN: 0025-5610, 1436-4646. DOI: 10.1007/s10107-004-0559-y. URL: http://link.springer.com/10.1007/s10107-004-0559-y (visited on 01/01/2026).

A Kinematics

For each segment, using homogeneous coordinates, the transformation is

$$T_{i-1}^{i}(\phi_i,\theta_i,s) = \begin{pmatrix} R_z(\phi_i) & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} R_y(\theta_i) & t_i(\theta_i,s) \\ 0 & 1 \end{pmatrix} \begin{pmatrix} R_z(-\phi_i) & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} R_z(\phi_i)R_y(\theta_i)R_z(-\phi_i) & R_z(\phi_i)t_i(\theta_i,s) \\ 0 & 1 \end{pmatrix}$$

where $t_i(\theta,s) = \begin{pmatrix} \rho(1-cos(\theta)) \\ 0 \\ \rho sin(\theta) \end{pmatrix} = \begin{pmatrix} \frac{s(1-cos(\theta))}{\theta} \\ 0 \\ \frac{ssin(\theta)}{\theta} \end{pmatrix}$.

We can explicitly write

$$R_z(\phi_i)R_y(\theta_i)R_z(-\phi_i) = \begin{pmatrix} \cos^2(\phi_i)(\cos(\theta_i)-1)+1 & \sin(\phi_i)\cos(\phi_i)(\cos(\theta_i)-1) & \cos(\phi_i)\sin(\theta_i) \\ \sin(\phi_i)\cos(\phi_i)(\cos(\theta_i)-1) & \cos(\theta_i)-\cos^2(\phi_i)(\cos(\theta_i)-1) & \sin(\phi_i)\sin(\theta_i) \\ -\cos(\phi_i)\sin(\theta_i) & -\sin(\phi_i)\sin(\theta_i) & \cos(\theta_i) \end{pmatrix}$$

and

$$R_z(\phi_i)t_i(\theta_i,s) = \begin{pmatrix} \cos(\phi_i)\frac{s(1-cos(\theta_i))}{\theta_i} \\ \sin(\phi_i)\frac{s(1-cos(\theta_i))}{\theta_i} \\ \frac{ssin(\theta_i)}{\theta_i} \end{pmatrix}$$

.

We get the transformation of the nth segment as $T_n(s) = \left(\prod_{i=1}^{n-1} T_{i-1}^{i}(L_i)\right) T_{n-1}^{n}(s), n > 1$ where s is the distance along the segment starting at its base. The kinematics of the tips are therefore $T_n(s = L_n)$.

B Dynamics

The dynamic is $M(q)\ddot{q} + (C(q,\dot{q}) + D(q,\dot{q}))\dot{q} + Kq + G(q) = J_{\text{tendon}}^T \tau_{\text{tendon}}$ with the following matrices:

$$\begin{cases} M = \int_0^L (\mu_{\text{displaced}} + \mu(s))J(s)^T J(s)ds \\ C = \int_0^L \mu(s)J(s)^T \dot{J}(s)ds \\ G = \int_0^L J(s)^T((\mu_{\text{buoyancy}} - \mu)\mathbf{g})ds,\ \mu := \text{mass per length} \\ J_{\text{tendon}} = \begin{pmatrix} J_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & J_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & J_n \end{pmatrix}, \text{ with } J_i = \begin{pmatrix} -\theta_i r_d \sin(\sigma_{3i} - \phi_i) & r_d \cos(\sigma_{3i} - \phi_i) \\ -\theta_i r_d \sin(\sigma_{3i+1} - \phi_i) & r_d \cos(\sigma_{3i+1} - \phi_i) \\ -\theta_i r_d \sin(\sigma_{3i+2} - \phi_i) & r_d \cos(\sigma_{3i+2} - \phi_i) \end{pmatrix} \end{cases}$$

r_d is the radius of the arm, σ contains the routing angle of each tendon. For 2 segments, evenly spaced tendons: $\sigma = \begin{bmatrix} \frac{\pi}{3} & \pi & \frac{5\pi}{3} & 0 & \frac{2\pi}{3} & \frac{4\pi}{3} \end{bmatrix}$

The water environment enters the dynamics through a buoyancy term, added mass and added drag : $\mathbf{D}_{\text{fluid}} = \sum_{i=1}^{N} \left[\frac{1}{2}\rho C_d A_i \left(\|\mathbf{J}_i\dot{\mathbf{q}}\|_2 + \epsilon\right)\right] \mathbf{J}_i^T \mathbf{J}_i$. we have $\mu_{\text{buoyancy}} = \mu_{\text{displaced}} = fill\rho_{\text{fluid}}\left(\pi r_o^2\right)$. The $fill$ term represents the infill of the arm $fill \in [0,1]$

C Code

The code is available on Github.